Sprint #1

Curso: Big Data

Proyecto: Retail Inteligente & AI Forecasting

Predicción de demanda y optimización de inventario

Integrantes:

- Isaac Ulloa Calvo
- Jean Carlo Ramirez Carranza
- Jeffrey Jimenez Cordero

CUC

I cuatrimestre 2026

Profesor:

Ericka Valverde Navarro

1. **Definir el problema**

Contexto (¿en qué industria y situación ocurre?)

Para el retail inteligente y AI Forecasting donde más se da es en empresas minoristas como por ejemplo Walmart, se usa IA para hacer retail inteligente y tener experiencia de cliente personalizada ya que la IA se usa para analizar datos como historiales de pedidos, historiales de navegación y programas de fidelización para mejorar la relevancia en marketing. También ayuda para tener una gestión de inventario automatizada para determinar el momento adecuado para colocar productos y también ayuda en prevención de demanda de productos.

Dolor o necesidad (¿qué está fallando o qué se quiere mejorar?)

Que en la actualidad hay empresas u organizaciones que no saben cómo exprimir el uso de IA y el valor de la data que tengan, están desaprovechando oportunidades, deben aprender a tener gestión orientada a objetivos usando IA y aprovechando data que tengan y en la actualidad se sabe que aún hay empresas pequeñas que gestionan sus ventas en papel, en lo que se falla es en la implementación de tecnología. por temas de costos o no se animan para innovar Impacto.

¿A quién afecta y por qué importa?

afecta de buena manera con beneficios para las empresas, como personalización de la experiencia del cliente y optimización de gestión de inventarios y es algo que importa debido a que se puede analizar datos en tiempo real gracias a la IA y así anticipar comportamientos con precisión y gracias al Forecast podemos analizar grandes volúmenes de datos.

Alcance (qué incluye y qué NO incluye el proyecto)

Este proyecto incluye el uso de datos históricos de ventas de alguna empresa u organización, vamos a utilizar python para limpieza de datos y análisis y luego con datos limpios podemos empezar a buscar insights o información valiosa para poder graficar los datos, hacer KPIs y demás visualizaciones para la comprensión del negocio y también implementar modelos de IA para hacer predicciones

Salida esperada:

El problema que se presenta es que las empresas tienen perdida de dinero por malos análisis y predicciones, llegando a tener sobrestock y un mal retail, esto conlleva pérdidas de dinero, clientes y una mala gestión de inventario obligando a la empresa a tener que hacer liquidaciones masivas para los productos con sobrestock.

2. Objetivo general

Este proyecto está hecho para mejorar el retail de empresas gracias al analisis de datos para poder tomar mejores decisiones basadas en datos

2.1 Objetivos específicos

1. Analizar el comportamiento de los clientes en diferentes fechas del mes.

2. Identificar cuales son los clientes frecuentes.

3. Identificar productos más comprados

4. Predecir demanda de productos

3. Preguntas del análisis

1. ¿Qué tipo de producto se vende más en los diferentes meses del año?

2. ¿Cuántas ventas totales se hacen por mes?

3. ¿Qué tipo de clientes son los más frecuentes ?

4. ¿Qué producto podría ser que tenga más demanda el próximo año?

5. Hay patrones de compra en diferentes meses?

4. **Definir los datos (Catálogo inicial)**

1ra fuente: Transacciones de ventas (POS)

Esta es la fuente principal que alimenta tanto el histórico (Batch) y la detección de quiebres en tiempo real (speed)

- Nombre de la fuente: Sistema de punto de venta (POS) /Logs de transacciones
- Tipo de dato: Estructurado
- Frecuencia: Streaming (Tiempo real para la capa speed) y (Batch diario para el histórico)
- Campos Principales (Variables)

1. Id_Transaccion (PK)
2. Fecha_Hora_Compra (Fecha y hora exacta: YYYY-MM-DD HH:MM:SS)
3. Id_Tienda (identificación exacta de la sucursal)
4. Sku_Producto (Codigo único del producto)
5. Cantidad_Vendida (Número de unidades)
6. Precio_Unitario (Precio venta real)
7. Descuento_Aplicado (Monto descontado si se aplica)
8. Id_Cliente (Identificador del cliente)
9. Método_pago (Efectivo, Tarjeta, Sinpe, etc.)
10. Stock_Restante (Nivel de inventario registrado tras la venta)

- Volumen estimado: Alto, ya que la naturaleza del retail con múltiples sucursales t miles de transacciones diarias
- Simulación: Se generará un dataset sintético (CSV) usando Python, creando patrones donde ciertos días por ejemplo las quincenas tengan mayor volumen de Cantidad Vendida

2da Fuente: Eventos Extremos y Calendario Comercial

Son los datos necesarios para las predicciones de demanda

- Nombre de la fuente: API de clima / Calendario Corporativo
- Tipo de dato: Estructurado / Semiestructurado
- Frecuencia: Batch (Diario/Semanal)
- Campos Principales (Variables):

1. Fecha (YYYY-MM-DD)
2. Id_Tienda (Para asociar el clima con una tienda en específico)
3. Tipo_Evento (Ej: “Partido Selección”, “Concierto”, “Black Friday”)
4. Feriado (Booleano: 1 es feriado, 0 si no es feriado)
5. Temperatura_Promedio (Grados Celsius pronosticados)
6. Precipitación_mm (Milímetros de lluvia pronosticados)
7. Nombre_Promocion (Nombre de la promoción activa si es que la hay)
8. Categoria_Afectada (categoría de productos en oferta si la hay)

- Volumen estimado: Bajo, son pocos registros (Uno por día por tienda)
- Simulación: Se creara un Excel/CSV marcando las fechas festivas de Costa Rica y asignando valores aleatorios de temperatura según la época

1ra Fuente: Transacciones (Ventas)

Aquí se registran los datos que salen de la caja registradora

-Que hace: Nos dice cuánto vendimos y que productos se fueron de la tienda

-Por que importa: Es la base para saber qué productos nos quedan y que tan bien le va al negocio

2da Fuentes : Eventos y clima (Contexto)

Son los datos de afuera que afectan al negocio, como si llueve o si es feriado

-Que hace: Nos dice que estaba pasando en el mundo cuando ocurrió la venta

-Por qué importa: Las ventas no ocurren en el vacío el clima y las fiestas (Factores Externos) cambian lo que la gente compra

La relación (El por qué)

Al unir las 2, le damos lógica a los números

La fuente 1 dice: “Hoy se vendió mucha carne”

La fuente 2 dice: “Es que hoy es feriado y hace sol”

conclusión: Juntas enseñan a la inteligencia artificial a entender que sol + feriado = Venta de carne

5. Definir usuarios y casos de uso

Usuario 1: Gerente de la tienda

Rol: Operativo

decisión: Este solicita el reabastecimiento urgente o mover la mercancía de la bodega a la estantería

Lo que necesita ver: Tener una alerta en su dispositivo móvil diciendo que Sku_Producto con Stock_Restante es bajo

Usuario 2: Planificador de demanda

Rol: Analista/ estratégico

decisión: Ver y analizar el volumen de las compras a los proveedores y la distribución logista a lo largo de los meses

Lo que necesita ver: Tener un dashboard con la predicción de Cantidad_Vendida futura por Sku_Producto y tener recomendaciones de compra

6.Proponer arquitectura Big Data

Arquitectura Lambda

Lambda porque permite combinar el procesamiento de grandes volúmenes de datos históricos (batch) con el análisis de datos recientes en tiempo casi real (speed), logrando predicciones de demanda más precisas y una mejor gestión del inventario.

6.1 Diagrama

```
┌──────────────────────────────────┐
│        1) FUENTES                │
│ Ventas | Promos | Inventario│
│ Eventos/Estacionalidad           │
└──────────────────────────────────┘
                │
┌──────────────────────────────────┐
│        2) INGESTA                │
│ Batch: CSV/JSON programado       │
│ Speed: eventos (micro-batch)     │
└──────────────────────────────────┘
                │
┌──────────────────────────────────────────────┐
│            3a) RAW / LANDING                 │
│ Data Lake local: CSV/JSON (crudo)            │
└──────────────────────────────────────────────┘
                │
┌──────────────────────────────────┐
│   4) PROCESAMIENTO BATCH         │
│ PySpark: limpiar, features│
│ agregaciones, entrenamiento│
└──────────────────────────────────┘
                │
┌──────────────────────────────────────────────┐
│          3b) CURATED / PROCESSED             │
│ Data Lake: Parquet (limpio/transformado)     │
└──────────────────────────────────────────────┘
                │
┌──────────────────────────────────┐
│   6) ANÁLISIS / IA               │
│ Prophet + scikit-learn           │
│ Predicción + productos crít│
└──────────────────────────────────┘
                │
┌──────────────────────────────────────────────────────────┐
│          5) SERVING (Servicio / Consulta)                │
│ PostgreSQL desnormalizado:                               │
│ predicciones | inventario_status | alertas               │
└──────────────────────────────────────────────────────────┘
                │
┌──────────────────────────────────────────────────────────┐
│                7) CONSUMO                                │
│ Streamlit dashboard | Alertas Telegram/Email             │
└──────────────────────────────────────────────────────────┘
```

Speed path (near real-time):
Ingesta Speed → (Spark Streaming / micro-batch) → PostgreSQL(alertas/estado)

7.Definir herramientas

7.1 Ingesta (Fuentes de datos): Python + Pandas (CSV/JSON)

Se utiliza Python junto con la librería Pandas para extraer y cargar datos provenientes de archivos CSV o JSON, ventas históricas, promociones e inventario, permite una limpieza inicial de los datos antes de su procesamiento.

7.2. Batch (Procesamiento Big Data): PySpark (Apache Spark)

PySpark se emplea para el procesamiento masivo de datos históricos en modo batch, permitiendo transformar grandes volúmenes de información, generar agregaciones y construir datasets preparados para el entrenamiento de modelos de predicción de demanda.

7.3 Speed (Procesamiento en tiempo casi real): Spark Streaming o micro-batch en Python

Para el procesamiento rápido de datos recientes se utiliza Spark Streaming o micro-batches en Python, permitiendo analizar eventos de ventas en intervalos cortos y detectar cambios relevantes en la demanda de productos.

7.4 Storage (Almacenamiento): Parquet + PostgreSQL desnormalizado
Almacenar información desnormalizada y facilitar consultas rápidas.

7.5 Forecast (Predicción de demanda): Prophet + scikit-learn

Construir modelos de predicción de demanda considerando estacionalidad, promociones y eventos externos.

7.6 Detección de productos críticos: Reglas de negocio + clasificación (scikit-learn)

La identificación de productos críticos se realiza mediante reglas de negocio y modelos de clasificación desarrollados con scikit-learn, una librería que permite analizar riesgos de quiebre o sobrestock.

7.8 Alertas automáticas: Python + Gmail

Las alertas se implementan utilizando Python junto con servicios gratuitos como correo electrónico o bots de Telegram, permitiendo notificaciones automáticas sin necesidad de plataformas de pago.

7.9 Dashboard (Visualización): Streamlit

Streamlit permite construir dashboards interactivos para visualizar predicciones, inventarios, alertas y recomendaciones de manera sencilla y accesible.

8. Supuestos

1. Existe historial suficiente (semanas/meses) para aprender estacionalidad y patrones.
2. Las ventas están bien fechadas y asociadas a un producto (ID consistente).
3. Las promociones y eventos externos se pueden vincular por fecha y producto/categoría.
4. El inventario disponible representa el stock real.
5. El volumen de datos es manejable para ejecución local.

8.1 Riesgos y mitigación

1. Calidad de datos baja (nulos, duplicados, errores)
2. Poco histórico o datos incompletos
3. Limitaciones de hardware local
4. Error humano, inventarios manuales
5. fluctuación en la demanda no predecible